
TECHNICAL ASSESSMENT · REMASTERING FEASIBILITY

Necoyoad

v10

*What was good, what was bad, what to keep,
and whether to rebuild from scratch with Laravel*

based on: 9 volumes, 216 pages of verified analysis
part I: 10 architectural strengths (good)
part II: 15 technical debts (bad)
part III: 8 patterns to preserve (keep)
part IV: Laravel mapping + 22-month effort estimate
pages: 16 · stance: prescriptive

Reverse-Engineered Architecture Report

Based on PHP source commit `ce75adf`
github.com/yosietserga/necoyoad

PRESCRIPTIVE · ASSESSMENT · RECOMMENDATIONS INCLUDED

Necoyoad — Technical Assessment & Remastering Feasibility (v10)

*What was good, what was bad, what to keep, and whether to rebuild
from scratch with Laravel or another modern framework*

Abstract

This is the tenth and final volume of the Necoyoad architecture blueprint. Unlike volumes v1–v9 (which were strictly descriptive), v10 is **prescriptive**: it evaluates the codebase, identifies what was well designed, what was poorly designed, what is worth preserving in a remastering effort, and assesses the feasibility of rebuilding the platform from scratch using Laravel (or a similar modern PHP framework).

The assessment is based on the exhaustive source-code analysis conducted across v1–v9: 1,450 PHP files, 87 database tables, 202 pages of verified architecture documentation. The findings are organised into four parts: **Part I — What Was Good** (10 architectural strengths worth preserving); **Part II — What Was Bad** (15 technical debts and design flaws); **Part III — What to Keep** (8 patterns and abstractions that should survive a remastering); and **Part IV — Feasibility of a Laravel Rebuild** (a pattern-by-pattern mapping from Necoyoad to Laravel, with effort estimates and a recommended migration strategy).

Keywords: Necoyoad, OpenCart, PHP 8.1, technical assessment, code review, remastering, Laravel, Symfony, migration feasibility, architecture patterns.

Contents

1	Introduction	4
1.1	From Descriptive to Prescriptive	4
1.2	Basis for Assessment	4
1.3	Context	4
I	What Was Good	4
2	Architectural Strengths	4
2.1	The AdminController Declarative CRUD Framework	4
2.2	The Polymorphic Object Spine	5
2.3	The Dual Hooks/Events Extension System	5
2.4	The Widget Composition System	6
2.5	The Per-Entity Template Override	6
2.6	The Banner jQuery Plugin Discriminator	6
2.7	The Multi-Store / Multi-Language Composition	6
2.8	The Campaign Marketing Pipeline	7
2.9	The Visual Theme Editor	7
2.10	The ntPlugins JavaScript Registry	7
II	What Was Bad	7
3	Technical Debt and Design Flaws	7
3.1	MyISAM Engine (No Transactions, No FKs)	7
3.2	Double-MD5 Password Hashing	8
3.3	The LIKE '%key=value%' Widget Query Pattern	8
3.4	N+1 Query Patterns	8
3.5	No API Authentication	8
3.6	Vestigial Bounce Processing	8
3.7	Inconsistent <code>object_to_store</code> Writes	9
3.8	Broken API PUT Branches	9
3.9	PHPMailer 5.0 from 2009	9
3.10	No Unsubscribe Mechanism	9
3.11	Route-Level RBAC Only	9
3.12	Hard-Coded Venezuelan Locale	10
3.13	<code>var_dump()</code> in Production Code	10
3.14	No File Versioning in CSS/TPL Editor	10
3.15	Double Link Rewriting in Campaigns	10
III	What to Keep	10
4	Patterns Worth Preserving in a Remastering	10
4.1	The Declarative Admin CRUD Pattern	10
4.2	The Polymorphic Object Spine	11
4.3	The Dual Hooks/Events System	11
4.4	The Widget Composition System	11
4.5	The Per-Entity Template Override	12
4.6	The Multi-Store/Multi-Language Architecture	12

4.7	The Campaign Marketing Pipeline	12
4.8	The Banner jQuery Plugin Discriminator	13
IV	Feasibility of a Laravel Rebuild	13
5	Pattern-by-Pattern Mapping	13
6	Effort Estimate	13
7	Recommended Migration Strategy	13
7.1	Option A: Full Rebuild with Laravel (Recommended)	13
7.2	Option B: Incremental Modernisation (Strangler Fig)	15
7.3	Option C: Remaster in Place (Modernise PHP, Keep Architecture)	15
8	Recommendation	15
9	What Would <i>Not</i> Survive a Rebuild	15
10	Conclusion	16

1 Introduction

1.1 From Descriptive to Prescriptive

Volumes v1–v9 were strictly descriptive: they documented how the system *is*, without proposing improvements. This volume changes stance. The user has asked for an evaluative assessment: what was good, what was bad, what can be reused, and whether to rebuild from scratch. v10 answers those questions directly.

1.2 Basis for Assessment

The assessment draws on:

- v1 (SQL-only reconstruction) — the 87-table schema.
- v2 (source verification) — the bootstrap, engine, auth, API.
- v3 (rendering pipeline) — the widget composition system.
- v4 (marketing subsystem) — the campaign pipeline.
- v5 (multi-store/multi-language) — the polymorphic object spine.
- v6 (admin back-office) — the AdminController declarative CRUD.
- v7 (menu links) — the recursive tree rendering.
- v8 (CMS posts/pages) — the widget composition modes.
- v9 (banner subsystem) — the jQuery plugin discriminator.

Total: 216 pages of verified architecture documentation, covering every subsystem of the platform.

1.3 Context

Necoyoad is a PHP 8.1 / MySQL 5.7 e-commerce and CMS platform, derived from OpenCart 1.x/2.x, with significant proprietary extensions. The codebase is ~17,000 lines of PHP (excluding vendor libraries), 1,450 files, 87 database tables. It was built for the Venezuelan market (RIF tax IDs, bank-transfer payments, Spanish-first UI) circa 2009–2023.

Part I

What Was Good

2 Architectural Strengths

2.1 The AdminController Declarative CRUD Framework

AdminController — the crown jewel of the admin

The `AdminController` base class (1,244 lines, v6) is a declarative CRUD framework that reduces 70+ admin controllers to a few lines of configuration each. A subclass declares: `$form_vars` (field definitions with types), `$filters` (list-view filter definitions), `$public_methods` (accessible CRUD methods), `$model_name/$model_route` (the model to load), and `$object_type` (the polymorphic discriminator).

The base provides the entire CRUD lifecycle: `index()` (list), `insert()/update()` (via shared `upsert()`), `copy()`, `delete()`, `activate()`, `sortable()`, `grid()` (AJAX grid), `getList()`, `getForm()`, `validateForm()`, `validateDelete()`.

This is a significant abstraction over vanilla OpenCart (where each controller has its own CRUD) and over many modern frameworks (where CRUD is typically generated by Artisan commands rather than declared at the class level). The declarative approach means adding a new admin entity is a matter of writing a controller with a few properties, not copy-pasting 500 lines of boilerplate.

2.2 The Polymorphic Object Spine

Five-table polymorphic spine — scalable and extensible

The five polymorphic tables (`object_to_store`, `object_to_category`, `description`, `property`, `url_alias`) use the (`object_id`, `object_type`) pair as a composite soft foreign key. This means:

- Adding a new content type (e.g. “survey”, “auction”) requires *no schema changes* — just populate `object_type` with the new string.
- Localisation is built-in: every content type gets per-language descriptions via the `description` table’s (`object_id`, `object_type`, `language_id`) triple.
- SEO URLs are built-in: every content type gets per-language slugs via `url_alias`.
- Free-form metadata is built-in: every content type gets EAV properties via the `property` table.
- Multi-store scoping is built-in: every content type gets store assignment via `object_to_store`.

This is a well-designed polymorphic architecture that anticipates future content types. It is more flexible than Laravel’s polymorphic relations (which require a morph map) because the discriminator is a free-form string, not a class name.

2.3 The Dual Hooks/Events Extension System

Hooks + Events — the right tool for each job

The dual extension system (v2, v3) provides:

- **Hooks** (WordPress-style): `addFilter`, `applyFilters`, `addAction`, `run`. Filters chain values through callbacks; actions short-circuit on truthy return. Used for `render`, `fetch`, `loadWidgets`, `csspath`, `module:settings`.
- **Events** (Node-style): `on`, `emit`, `off`. No short-circuit; all listeners fire. Used for `dispatch`, `beforeLoad`, `afterLoad`, `registry:update`, `php:error`.

The separation is architecturally sound: Hooks are for *transforming* values and *short-circuiting* flows (I want to change the rendered HTML); Events are for *broadcasting* occurrences (I want to know when a dispatch happens). Many modern frameworks conflate these into a single event system, which leads to confusion about whether a listener can modify the flow.

2.4 The Widget Composition System

Three-mode widget composition — flexible and powerful

The widget composition system (v3, v8) supports three modes:

1. **Dynamic:** admin configures widgets via the visual editor; `widgets-rows.tpl` emits `{%widget_name%}` tokens; `fetch()` substitutes them.
2. **Manual:** template author hardcodes `{%widget_name%}` tokens in a `.tpl` file.
3. **Hybrid:** combines dynamic sections with hardcoded sections.

Plus: per-entity widget overrides (product-specific widgets), per-item widget composition (banner slide overlays), the **only:** prefix for embedded pages, and the `widgets-common.tpl` universal layout. This is a page-builder experience that rivals WordPress's block editor or Drupal's Layout Builder, implemented in raw PHP.

2.5 The Per-Entity Template Override

Per-entity template override via EAV

Each individual post, page, product, or category can use a different template, set via the admin's visual editor and stored in the EAV property('style', 'view') key. The resolution is 3-level: EAV property → config default → hardcoded fallback. This allows a blog post to use a standard template, a landing page to use a custom template with SVG sections, and a product page to use a custom product-info template — all without code changes.

2.6 The Banner jQuery Plugin Discriminator

jquery-plugin discriminator — plugin-aware widget

The `banner.jquery_plugin` column (v9) drives the entire rendering pipeline: template selection, JS enqueueing, CSS enqueueing, and the `ntPlugins` registry push. This is the only widget module that dynamically selects its template from a database column. The 33-template library (NivoSlider, Slick, Camera, Fancybox, etc.) with per-template JS/CSS assets is a rich visual content system.

2.7 The Multi-Store / Multi-Language Composition

Store x Language matrix — data-level multi-tenancy

The multi-store architecture (v5) is a clean data-level multi-tenancy: `STORE_ID` as a global constant, filtered at query time by every model. The 6-level language detection priority chain (GET → session → cookie → browser → config) is thorough. The $S \times L$ composition matrix (5 filtering dimensions at query time) produces a unique content surface per (store, language) pair without a dedicated schema construct.

2.8 The Campaign Marketing Pipeline

Self-hosted Mailchimp clone

The marketing subsystem (v4) is a complete email-marketing pipeline: newsletter composition with WYSIWYG, contact-list targeting, scheduled sending via cron, per-recipient personalisation (7 tokens + 13 store-level tokens), open/click tracking with full HTTP context, and a 50-email-per-pass throttle with 15-minute defer. This is more than many commercial e-commerce platforms offered in 2009, and it is fully self-hosted.

2.9 The Visual Theme Editor

Visual theme editor with nt-editable hooks

The visual theme editor (v3) is built into the HTML itself via `nt-editable`, `data-widget`, `data-position`, `data-necotienda_module` attributes. The editor's JS scans the DOM and attaches drag/click handlers. The `/**key**/ ... /**/key**/` CSS markers allow per-widget CSS editing. The `theme_style` table stores CSS-rule overrides at the selector/property/value granularity. This is a no-code customisation experience.

2.10 The ntPlugins JavaScript Registry

ntPlugins — decoupled deferred initialisation

The `ntPlugins` registry (v9) is a decoupled, deferred- initialisation pattern for jQuery plugins. Each slider template pushes a `{id, config, plugin}` entry, and `loadNTPlugins()` applies the plugin on DOM ready. This allows multiple sliders on the same page without templates knowing about each other — a clean observer pattern for the client side.

Part II

What Was Bad

3 Technical Debt and Design Flaws

3.1 MyISAM Engine (No Transactions, No FKs)

MyISAM — the root cause of many issues

All 87 tables use MyISAM. This means: no transactions (multi-table writes like order creation are not atomic), no foreign keys (all `_id` columns are soft references, referential integrity is the application's responsibility), table-level locking (concurrent writes to `stat`, `campaign_stat`, `search` block each other), and no crash recovery. This is the single biggest technical debt in the codebase.

3.2 Double-MD5 Password Hashing

Double-MD5 with per-row salt — insecure by modern standards

The password algorithm (v2) is double-MD5 with a per-row random salt: `md5(md5(plaintext) . suffix) . ':' . suffix`. While the salt prevents rainbow-table attacks, MD5 is computationally cheap (making brute-force feasible) and the double-MD5 offers no meaningful security improvement over single-MD5. Modern standards require `bcrypt`, `argon2id`, or `scrypt` — slow hashes designed to make brute-force infeasible.

3.3 The LIKE '%key=value%' Widget Query Pattern

LIKE pattern — full-table scans on every widget lookup

Every widget filter (v3, v5) is a `LIKE` query against the serialised settings blob: `WHERE p.value LIKE ...`. This forces a full-table scan on every widget lookup. On a page with 5 layout positions, each with 3 rows, each with 2 columns, each with 4 widgets, that is 165 full-table scans. The cache absorbs most of the cost, but the first request (or any request after a cache clear) is extremely slow.

3.4 N+1 Query Patterns

Recursive query explosion in widget and menu trees

The recursive `getLinks()` method (v7) and the per-object widget query (v5) both exhibit N+1 patterns: one query per parent, one query per child, one query per widget. A 3-level menu with 5/3/2 items generates 63 queries on cache miss. The widget system's two-query merge (default tree + per-object tree) doubles the query count for every layout position.

3.5 No API Authentication

`validateTokens()` stub — the API is fully open

The admin API's `validateTokens()` method (v2) is a stub that always returns `true`. The TODO comment acknowledges that public/private API keys, tokens, expiry, access level, licensing, and premium membership checks were planned but never implemented. The only protection is the admin session token pre-action. Any caller with an admin session can hit every API endpoint.

3.6 Vestigial Bounce Processing

Bounce processor is dead code

The bounce processor (v4) is commented out of the cron and references non-existent Inter-spire SendStudio classes. There is no working bounce detection. Bounced emails are not automatically unsubscribed; the contact stays active and will be emailed again on the next campaign.

3.7 Inconsistent `object_to_store` Writes

Legacy and polymorphic columns coexist in the same table

The admin store model (v5) writes products, categories, and manufacturers using legacy column names (`product_id`, `category_id`, `manufacturer_id`), but writes downloads, pages, and banners using the polymorphic pair (`object_id`, `object_type`). The storefront reads using the polymorphic pair. Posts and post-categories `INSERT` statements omit `store_id` entirely. This inconsistency means some content types are not correctly store-scoped.

3.8 Broken API PUT Branches

All 5 marketing API PUT branches are broken

The PUT (update) branches of all 5 marketing API endpoints (v4) use the loaded DB row as the data source instead of the JSON body, or reference an undefined `$query` variable (mailservers). The GET, POST, and DELETE branches work. Only PUT is broken.

3.9 PHPMailer 5.0 from 2009

PHPMailer 5.0 — 14 years old and insecure

The campaign sender (v4) uses PHPMailer 5.0 (released 2009), renamed `Mailer`. It supports only `AUTH LOGIN` (no `OAuth2`, no `CRAM-MD5`). Multiple CVEs have been disclosed against PHPMailer 5.x over the years. The SMTP client is a from-scratch implementation, not a wrapper around PHP's `mail()`.

3.10 No Unsubscribe Mechanism

No unsubscribe — CAN-SPAM/GDPR non-compliance

There is no `?unsubscribe=1` endpoint and no `List-Unsubscribe` email header (v4). Recipients cannot opt out of campaigns. This is non-compliant with CAN-SPAM (US), GDPR (EU), and most modern email regulations.

3.11 Route-Level RBAC Only

No object-level permissions

The admin RBAC (v6) checks whether the route (e.g. `store/product`) is in the user group's `access/modify` arrays. It does not check whether the user has permission to edit a *specific* product. Any admin with `modify` permission on `store/product` can edit any product in any store.

3.12 Hard-Coded Venezuelan Locale

Venezuelan locale hard-coded throughout

America/Caracas is hard-coded in the campaign form, the storefront `link2Campaign`, and the `Cron` constructor (v4). The subdomain regex is hard-coded to `necoyoad.com` (v5). The tracking pixel is hard-coded to `necoyoad.com/.../firma_necoyoad.jpg` (v4). These make the platform difficult to deploy for non-Venezuelan markets.

3.13 `var_dump()` in Production Code

`var-dump` left in the links widget

The `ControllerModuleLinks::getLinks()` method (v7, line 56) contains a `var_dump($descriptions)` call that fires when `submenu_type === 'html_content'`. Every menu link with an HTML-content submenu dumps the descriptions array to the browser.

3.14 No File Versioning in CSS/TPL Editor

CSS/TPL editor has no backup or versioning

The `style/editor.php` controller (v6) lets the admin edit CSS and TPL files directly with `file_put_contents()`. There is no backup, no versioning, no diff. A typo in a CSS file can break the storefront with no recovery path short of a server backup.

3.15 Double Link Rewriting in Campaigns

Per-recipient link rewriting happens twice

The campaign commit phase (v4) rewrites links and inserts `campaign_link` rows. The cron sender rewrites them *again* and inserts fresh rows with new nonces. The commit-phase rows are dead data if the per-recipient cache hits. The `campaign_link` lookup is not scoped by `campaign_id`, so a nonce from one campaign can resolve to a redirect from another.

Part III

What to Keep

4 Patterns Worth Preserving in a Remastering

4.1 The Declarative Admin CRUD Pattern

AdminController declarative pattern → Laravel Form Requests + Resource Controllers

The AdminController's declarative approach (`$form_vars`, `$filters`, `$public_methods`) is worth preserving. In Laravel, this maps to:

- `$form_vars` → Form Request validation rules (with type inference).
- `$filters` → Query scopes on the Eloquent model.

- `$public_methods` → Middleware-based route permission checks.
- `$model_route` → Route model binding.
- `$object_type` → Eloquent morph map.

A base `AdminController` class could still provide `index()`, `store()`, `update()`, `destroy()`, `grid()` — but leveraging Laravel’s Form Requests, Resource Controllers, and Eloquent instead of raw SQL.

4.2 The Polymorphic Object Spine

Polymorphic spine → Laravel Polymorphic Relations

The five polymorphic tables map directly to Laravel’s polymorphic relations:

- `description` → `MorphMany` on each model.
- `property` → `MorphMany` with a `group` + `key` composite.
- `object_to_store` → `MorphToMany` (Store model).
- `object_to_category` → `MorphToMany` (Category model).
- `url_alias` → `MorphMany` with a `language_id` scope.

The morph map would register each content type explicitly (`'product' => Product::class`, etc.), which is cleaner than the free-form string discriminator but loses some flexibility.

4.3 The Dual Hooks/Events System

Hooks + Events → Laravel Events + Filters

Laravel’s event system maps to Necoyoad’s Events (observer pattern). For the Hooks (filter/action with short-circuit), Laravel doesn’t have a direct equivalent, but the pipeline pattern (middleware) or a custom `Filter` facade could replicate it. Alternatively, Laravel’s `Event::dispatch()` with return values (Laravel 9+) can serve as both observer and filter.

4.4 The Widget Composition System

Widget composition → Laravel Blade Components + View Composers

The widget composition system maps to Laravel's Blade components:

- Each widget becomes a Blade component (`<x-widgets.banner>` with settings prop).
- The `{%widget_name%}` placeholder becomes a Blade slot or a dynamic component (`@stack('widgets')` or a dynamic component).
- The `loadWidgets()` method becomes a View Composer that populates the view with widget data.
- The per-entity widget override becomes a scoped View Composer that checks the session's `object_type`.
- The visual editor's `nt-editable` attributes can be preserved with a Blade directive (`@editable('widget', $name)`).

4.5 The Per-Entity Template Override

Per-entity template override → Laravel dynamic view resolution

Laravel's `View::exists()` + `view()` can replicate the 3-level template resolution:

```
$template = $entity->properties['style']['view'] ?? null;
$default = config("defaults.{$entity->type}", "content/{$entity->type}");
$template = $template ?: $default;
if (view()->exists("themes/{$theme}/{$template}")) {
    return view("themes/{$theme}/{$template}", $data);
} else {
    return view("themes/choroni/{$template}", $data);
}
```

4.6 The Multi-Store/Multi-Language Architecture

Multi-store → Laravel multi-tenancy + localization

Laravel has mature multi-tenancy packages (e.g. `tenancy/tenancy`, `stanc1/tenancy`) that provide database-level or row-level multi-tenancy. The row-level approach (`store_id` on every table) maps to a global scope that filters by the current tenant. Laravel's built-in localization (`App::setLocale()`, translatable packages like `spatie/laravel-translatable`) handles the language dimension.

4.7 The Campaign Marketing Pipeline

Campaign pipeline → Laravel Queues + Mail + Horizon

The campaign pipeline maps cleanly to Laravel's queue infrastructure:

- `task_queue` → Laravel Queue (Redis/Database driver).
- `CronSend::sendCampaign()` → a queued Job (`SendCampaignEmail`).
- `task_exec` locking → Laravel's built-in job locking (`ShouldBeUnique`).
- PHPMailer 5.0 → Laravel Mail (Symfony Mailer under the hood, with OAuth2 support).

- 50-email throttle → Laravel's `RateLimited` middleware.
- Tracking pixel + link redirect → controller routes with `DB::insert()` on each hit.
- Bounce processing → Laravel's `Mail::failure()` callbacks + a bounce-handling job.

4.8 The Banner jQuery Plugin Discriminator

jquery-plugin discriminator → Laravel dynamic Blade components

The `jquery_plugin` discriminator maps to Laravel's dynamic Blade components: `<x-dynamic-component>` with a component prop. Each slider becomes a Blade component with a component prop. Each slider becomes a Blade component (`resources/views/components/sliders/nivo-slider.blade.php`). The `ntPlugins` registry can be replaced by Alpine.js or Livewire for client-side initialisation.

Part IV

Feasibility of a Laravel Rebuild

5 Pattern-by-Pattern Mapping

6 Effort Estimate

7 Recommended Migration Strategy

7.1 Option A: Full Rebuild with Laravel (Recommended)

1. Start with a fresh Laravel 11 project.
2. Migrate the database to InnoDB with proper foreign keys.
3. Implement the polymorphic object spine as Eloquent traits.
4. Build the AdminController base + Form Request pattern.
5. Port the 70+ admin entities (mostly declarative, should be fast).
6. Build the widget composition system as Blade components + View Composers (the hardest part).
7. Port the storefront (catalog, checkout, account).
8. Port the CMS (posts, pages).
9. Port the marketing pipeline as Queue jobs.
10. Port the banner subsystem as dynamic Blade components.
11. Build the visual theme editor with Livewire/Inertia.
12. Build the API with Sanctum + API Resources.
13. Deploy with Docker + CI/CD.

Pros: clean codebase, modern framework, proper testing, security best practices, active community. **Cons:** 22 developer-months of effort, no incremental revenue during the rebuild.

Table 1: Necoyoad → Laravel pattern mapping.

Necoyoad pattern	Laravel equivalent	Effort
Front Controller + Action Registry (service locator)	Laravel Routing + Controllers	Low — Laravel’s router is more capable.
AdminController	Laravel Service Container	Low — automatic DI.
declarative CRUD	Form Requests + Resource Controllers	Medium — needs a base controller.
Polymorphic object spine	Eloquent Morph Relations	Medium — morph map per type.
Hooks (WordPress-style)	Custom Filter facade or Pipeline	Medium — no built-in equivalent.
Events (Node-style)	Laravel Events + Listeners	Low — direct mapping.
Widget composition	Blade Components + View Composers	High — core feature, needs careful design.
{%widget_name%} substitution	Blade slots / dynamic components	Medium.
Per-entity template override	view()->exists() + dynamic resolution	Low.
Multi-store (data-level)	Multi-tenancy package + global scope	Medium — tenant resolution middleware.
Multi-language (6-level priority)	Laravel Localization + middleware	Low.
Campaign pipeline	Laravel Queues + Mail + Horizon	Medium — job design + tracking.
Banner jquery_plugin	Dynamic Blade components	Low.
ntPlugins JS registry	Alpine.js / Livewire / JS modules	Medium — front-end rewrite.
SEO URL rewriter	Route Model Binding + slug column	Low.
MyISAM tables	MySQL InnoDB + migrations	Medium — schema migration.
Double-MD5 passwords	bcrypt / argon2id	Low — migration hash check.
Cron CLI	Laravel Scheduler + Queue Worker	Low.
Visual theme editor	Livewire / Inertia + contenteditable	High — significant front-end work.
deps.php asset manifest	Laravel Vite + manifest	Low.

Table 2: Estimated effort by subsystem (senior developer-months).

Subsystem	Effort	Notes
Core framework (routing, DI, auth)	1 mo	Mostly leveraging Laravel builtins.
Database migration (InnoDB + schema)	2 mo	87 tables, data migration, FK constraints.
Admin CRUD framework	2 mo	Base controller + Form Requests + 70+ entities.
Widget composition system	3 mo	The hardest part — Blade components, view composers, visual editor.
Storefront (catalog, checkout, account)	3 mo	Product, category, cart, order, customer.
CMS (posts, pages, categories)	1 mo	Simpler than e-commerce.
Marketing pipeline	2 mo	Queue jobs, mail, tracking, bounce.
Banner subsystem	1 mo	Dynamic components, 33 slider templates.
Multi-store / multi-language	1 mo	Tenancy package + localization.
Visual theme editor	3 mo	Significant front-end (Livewire/Inertia + drag-and-drop).
API (REST, with proper auth)	1 mo	Laravel Sanctum + API Resources.
Testing + deployment	2 mo	PHPUnit, CI/CD, Docker.
Total	22 mo	~2 developer-years (senior).

7.2 Option B: Incremental Modernisation (Strangler Fig)

1. Keep the existing codebase running.
2. Add Laravel alongside (via `require` in `composer.json`).
3. Migrate one subsystem at a time (start with the API, then admin, then storefront).
4. Use Laravel's database connection to access the existing MyISAM tables (read-only initially, then read-write as each subsystem is migrated).
5. Migrate the database to InnoDB table-by-table.
6. Eventually decommission the old codebase.

Pros: incremental, no big-bang risk, revenue continues. **Cons:** longer total timeline (~30 months), complexity of running two codebases.

7.3 Option C: Remaster in Place (Modernise PHP, Keep Architecture)

1. Migrate MyISAM → InnoDB.
2. Replace double-MD5 with `bcrypt`.
3. Fix the `object_to_store` inconsistencies. Fix the broken API PUT branches.
4. Add unsubscribe mechanism.
5. Replace PHPMailer 5.0 with PHPMailer 6.x.
6. Fix the `var_dump` and other bugs.
7. Add PHPUnit tests.

Pros: lowest effort (~6 developer-months), keeps the working codebase. **Cons:** doesn't address the fundamental architecture issues (MyISAM, no DI container, raw SQL, no testing culture).

8 Recommendation

Option A (Full Rebuild with Laravel) is recommended if the goal is a modern, maintainable platform that can be extended for the next 10 years. The 22-month effort is significant but the result is a clean codebase with proper testing, security, and community support.

Option B (Strangler Fig) is recommended if there is a running business that cannot afford a big-bang rebuild. The incremental approach reduces risk and allows revenue to continue, at the cost of a longer timeline and the complexity of running two codebases.

Option C (Remaster in Place) is recommended as a short-term stabilisation step before Option A or B. The 6-month effort fixes the most critical security and data-integrity issues without disrupting the business.

9 What Would *Not* Survive a Rebuild

- The raw SQL query builder (`Model::buildSQLQuery()`) — replaced by Eloquent.
- The `LIKE '%key=value%'` widget query pattern — replaced by proper JSON columns or a junction table.
- The `{%widget_name%}` string substitution — replaced by Blade slots.
- The MyISAM tables — replaced by InnoDB.

- The double-MD5 password hash — replaced by bcrypt.
- The `validateTokens()` stub — replaced by Sanctum.
- The PHPMailer 5.0 wrapper — replaced by Laravel Mail.
- The `.htaccess`-based routing — replaced by Laravel’s router.
- The `map.php` 843-line switch statement — replaced by Laravel’s autoloading + service provider pattern.
- The hand-rolled session handler — replaced by Laravel’s session driver.
- The file-based cache — replaced by Redis.

10 Conclusion

Necoyoad is a characterful, ambitious platform that was clearly built with care and a strong vision. The AdminController declarative CRUD framework, the polymorphic object spine, the dual Hooks/Events system, and the widget composition system are genuinely innovative — they anticipate patterns that modern frameworks are only now adopting (Laravel’s dynamic components, WordPress’s block editor, Drupal’s Layout Builder).

The technical debt (MyISAM, double-MD5, raw SQL, no testing) is significant but not unusual for a platform of this vintage (2009–2023). The bugs (broken API PUTs, vestigial bounce, `var_dump` in production, missing `store_id`) are fixable but indicative of a codebase that grew faster than its testing infrastructure.

A Laravel rebuild is feasible and recommended. The architecture maps well: the polymorphic spine maps to Eloquent morph relations, the widget system maps to Blade components, the AdminController maps to Form Requests, the marketing pipeline maps to Queue jobs. The 22-month effort estimate is for a senior developer; a team of 2–3 could complete it in 8–12 months.

The result would be a modern, testable, secure e-commerce + CMS platform that preserves the best of Necoyoad’s architecture while leaving behind its technical debt.